第四部分

算法设计策略

主讲人：孙歧峰

联系电话：13864717907
Q Q号码：274342509
邮　　箱：sunqf@upc.edu.cn

算法设计策略

算法设计是一种**创新**，更象一门艺术，没有一套规则可遵循，但是人们已经探索了一些行之有效的算法设计策略。

- 朴素策略：
  - 蛮力法—搜索和遍历
- 高级策略：
  - 分治策略（分而治之）
  - 贪心策略（优选策略）
  - 动态规划策略
  - 回溯策略
  - 分支界定策略
  - ……

算法设计策略——分治策略

问题分解易求解复杂问题时很自然的做法，将复杂问题分解成若干较小的问题求解易，个

好主意

越容易

这些子

，

且

的。

分治法的设计思想是，将一个难以直接解决的大问题，分割成一些规模较小的相同问题，以便各个击破，分而治之。

“凡治众如治寡，分数是也。”

——孙子兵法

不用再分就可求解

算法设计策略——分治策略

一、基本的设计思想

为了解决一个大（复杂）问题，可以：

1）分解：把它分解成两个或多个更小的问题；—划分步

2）解决：分别解决每个小问题；—治理步

3）合并：把各个子问题的解采用某种方式组合起来，得到原问题的解；—组合步

注意： 若分解后的子问题仍然很大，对子问题可以继续应用分而治之的技术。

算法设计策略——分治策略

形式化描述为：

```
Divide_and_Conquer(P)
{    if (|P|<=n0)    //|p|=n表示问题的规模，n0为一临界值
       return(ADHOC(P) );   //解决小问题的基本算法
     //分之
     else divide P into smaller subinstance P1,P2,...Pk;
     for (i=1;i<=k;i++)
        yi=Divide_and_Conquer(Pi);  //治之
     t=MERGE(y1,y2,...yk);   //组合而得到解
     return(t) }
```

其中：

|P|表示问题P的规模；n_0为一阈值，表示当问题P的规模不超过n_0时，问题已容易直接解出，而不必再继续分解。

ADHOC(P)是该分治法中的基本子算法，用于直接解小规模的问题P。因此，当P的规模不超过n_0时，直接用算法ADHOC(P)求解。

MERGE($y_1,y_2,...,y_k$)是该分治法中的合并子算法，用于将P的子问题$P_1,P_2,...,P_k$的相应的解$y_1,y_2,...,y_k$合并为问题P的解T。

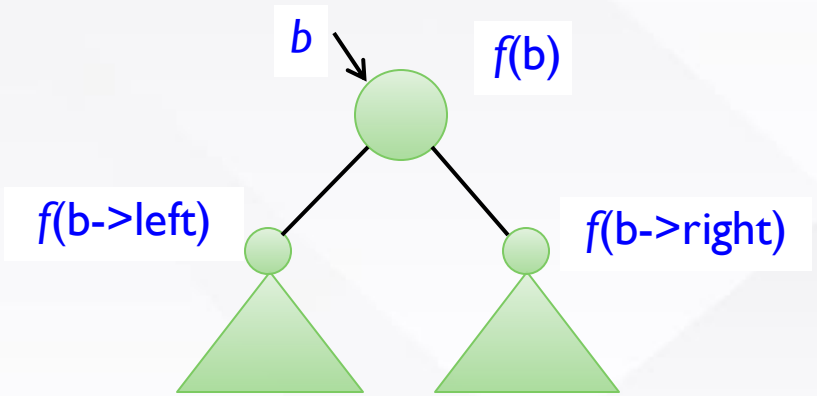


$T(n)=$ 分解问题的时间+解决每个小问题时间+组合解的时间 $=\sum T(n_i)+f(n)$

算法设计策略——分治策略

【例4-1】给定一棵采用二叉链存储的二叉树b，设计一个算法求其中叶子结点的个数。

解 设$f(b)$用于求二叉树b中叶子结点的个数



$f(b)=0$ 当b=NULL

$f(b)=1$ 当b为叶子结点

$f(b)=f(b->left)+f(b->right)$ 其他

参考算法：

```
int leafnodes(TreeNode* b)
{   if(b==NULL)              //空树
        return 0;
    if(b->left==NULL && b->right==NULL)
        return 1;            //只有一个叶子结点
    else                     //其他情况
        return leafnodes(b->left)+leafnodes(b->right);
}
```

算法设计策略——分治策略

可以看出分治法的设计由下面三个步骤组成：

1. 划分步。把输入的问题实例划分为k个子问题。一般来说应尽量使k个子问题的规模大致相同。在很多情况下，使k=2，也有k=1的划分，但是仍然把问题划分为两部分，取其中的一部分，而丢弃另一部分，例如二分检索问题，如果采用分治来处理，就可以这样处理。

根据分治法的分割原则，原问题应该分为多少个子问题才较适宜？各个子问题的规模应该怎样才为适当？这些问题很难予以肯定的回答。

但人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。换句话说，将一个问题分成大小相等的k个子问题的处理方法是行之有效的。许多问题可以取k=2。这种使子问题规模大致相等的做法是出自一种*平衡(balancing)子问题*的思想，它几乎总是比子问题规模不等的做法要好。

算法设计策略——分治策略

2. 治理步。当问题的规模大于某个预定义的阈值n_0时，治理步由k个递归调用组成。可以把阈值置为任何正常数，但是一般来说，阈值的大小经常与算法的性能有关。

对于某些算法，当k=2时，使阈值n_0=8或者16，有时可以改善算法的性能；而继续增大，使其达到某一点以后，算法的性能就开始下降。这在很大程度上取决于算法中的ADHOC对阈值n_0的敏感度，以及MERGE的处理情况。但是，应该强调，在某些算法里，阈值可能不会像1那么小，它必须大于某个常数。通过仔细地分析算法，通常可以找到这个常数。

算法设计策略——分治策略

3. 组合步。组合步把各个子问题的解组合起来。有些问题的合并方法比较明显，有些问题合并方法比较复杂，或者是有多种合并方案，或者是合并方案不明显，究竟应该怎样合并，没有统一的模式，需要具体问题具体分析。

组合步对分治算法的性能至关重要，算法的有效性很大程度地依赖于组合步的实现。例如，假定$n=k^m$，$n_0=1$；ADHOC解规模为1的问题，需花费1个单位时间；在把问题划分为k个子问题求解后，MERGE把k个子问题的解合并成原问题的解，需花费bn个单位时间，其中b是某个正常数。那么，分治算法的运行时间可由下面的递归方程确定：

$$\begin{cases} f(1)=1 & n=1 \\ f(n)=kf(n/k)+bn & n>1 \end{cases}$$

算法设计策略——分治策略

因为$n=k^m$,所以，有$f(k^m)=kf(k^{m-1})+bk^m$

采用替换，令$f(k^m)=h(m)$,上面的递归方程可以改写为：

$$\begin{cases} h(0)=1 & n=1 \\ h(m)=kh(m-1)+bk^m & n>1 \end{cases}$$

解这个递归方程，得到：

$$\begin{aligned} f(n)=f(k^m)&=h(m)=k(kh(m-2)+bk^{m-1})+bk^m \\ &=k^2h(m-2)+2bk^m \\ &=\cdots\cdots \\ &=k^mh(0)+mbk^m \\ &=n+bn\log_k n \end{aligned}$$

由上式可以看到，算法的整个运行时间中，ADHOC子算法共花费n个单位时间，而组合步花费的时间为$bn\log_k n$时间。当b>1，n很大时，它确定了算法的实际性能。

二、递归

递归是分治的典型，或者说分治产生了递归方法。

1、什么是递归

一个事物或者对象，部分地由其自身来定义或者描述，就称作是递归的。

2、递归算法和递归函数

直接或间接地调用自身的算法称为递归算法。用函数自身给出定义的函数称为递归函数。

3、递归和分治

由分治法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。在这种情况下，反复应用分治手段，可以使子问题与原问题类型一致而其规模却不断缩小，最终使子问题缩小到很容易直接求出其解。这自然导致递归过程的产生。

二、递归

分治与递归像一对孪生兄弟，经常同时应用在算法设之中，并由此产生许多高效算法。在计算机算法设计与分析中，使用递归技术往往使函数的定义和算法的描述简洁且易于理解。

例1：　阶乘函数

阶乘函数可递归地定义为：

$$n!=\begin{cases}1 & n=0\\ n(n-1)! & n>0\end{cases}$$

边界条件

递归方程

边界条件与递归方程是递归函数的两个要素，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。

算法设计策略——分治策略

例2　Fibonacci数列，无穷数列1，1，2，3，5，8，13，21，34，55，…，被称为Fibonacci数列。它可以递归地定义为：

$$F(n)=\begin{cases}1 & n=0\\ 1 & n=1\\ F(n-1)+F(n-2) & n>1\end{cases}$$

边界条件

递归方程

第n个Fibonacci数可递归地计算如下：

```
public static int fibonacci(int n)
  {
      if (n <= 1) return 1;
      return fibonacci(n-1)+fibonacci(n-2);
  }
```

例3　Ackerman函数

当一个函数及它的一个变量是由函数自身定义时，称这个函数是双递归函数。例如：Ackerman函数A(n，m)定义如下：

$$\begin{cases} A(1,0)=2 & \\ A(0,m)=1 & m\geq 0 \\ A(n,0)=n+2 & n\geq 2 \\ A(n,m)=A(A(n-1,m),m-1) & n,m\geq 1 \end{cases}$$

前2例中的函数都可以找到相应的非递归方式定义：

例1： $n!=1\cdot 2\cdot 3\cdot\cdots\cdot(n-1)\cdot n$

例2： $F(n)=\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{n+1}-\left(\frac{1-\sqrt{5}}{2}\right)^{n+1}\right)$

但本例中的Ackerman函数却无法找到非递归的定义。

算法设计策略——分治策略

例4　排列问题：设计一个递归算法生成n个元素$\{r_1,r_2,\ldots,r_n\}$的全排列。

- 设$R=\{r_1,r_2,\ldots,r_n\}$是要进行排列的n个元素，$R_i=R-\{r_i\}$。
- 集合X中元素的全排列记为perm(X)。
- (r_i)perm(X)表示在全排列perm(X)的每一个排列前加上前缀得到的排列。于是：

R的全排列可归纳定义如下：

当n=1时，perm(R)=(r)，其中r是集合R中唯一的元素；

当n>1时，perm(R)由$(r_1)perm(R_1)$，$(r_2)perm(R_2)$，…，$(r_n)perm(R_n)$构成。

算法设计策略——分治策略

例5 整数划分问题.

将正整数n表示成一系列正整数之和:

$n=n_1+n_2+\cdots+n_k$， 其中$n_1 \geqslant n_2 \geqslant \cdots \geqslant n_k \geqslant 1$，$k \geqslant 1$。

正整数n的这种表示称为正整数n的划分。求正整数n的不同划分的个数。

例如正整数6,则有如下11种不同的划分:

6;
5+1;
4+2，4+1+1;
3+3，3+2+1，3+1+1+1;
2+2+2，2+2+1+1，2+1+1+1+1;
1+1+1+1+1+1。

算法设计策略——分治策略

前面的几个例子中，问题本身都具有比较明显的递归关系，因而容易用递归函数直接求解。

但在本例中，如果设p(n)为正整数n的划分数，则难以找到递归关系，因此考虑增加一个自变量：

将最大加数n_1不大于m的划分个数记作q(n,m)。可以建立q(n,m)的如下递归关系：

(1) q(n,1)=1,n≥1;

当最大加数n_1不大于1时，任何正整数n只有一种划分形式，即： $n = \overbrace{1+1+\ldots+1}^{n}$

(2) q(n,m)=q(n,n),m≥n;

最大加数n_1实际上不能大于n。因此，q(1,m)=1。

(3) q(n,n)=1+q(n,n-1);

正整数n的划分由n_1=n的划分和n_1≤n-1的划分组成。

(4) q(n,m)=q(n,m-1)+q(n-m,m),n>m>1;

正整数n的最大加数n_1不大于m的划分由n_1=m的划分和n_1≤m-1 的划分组成。

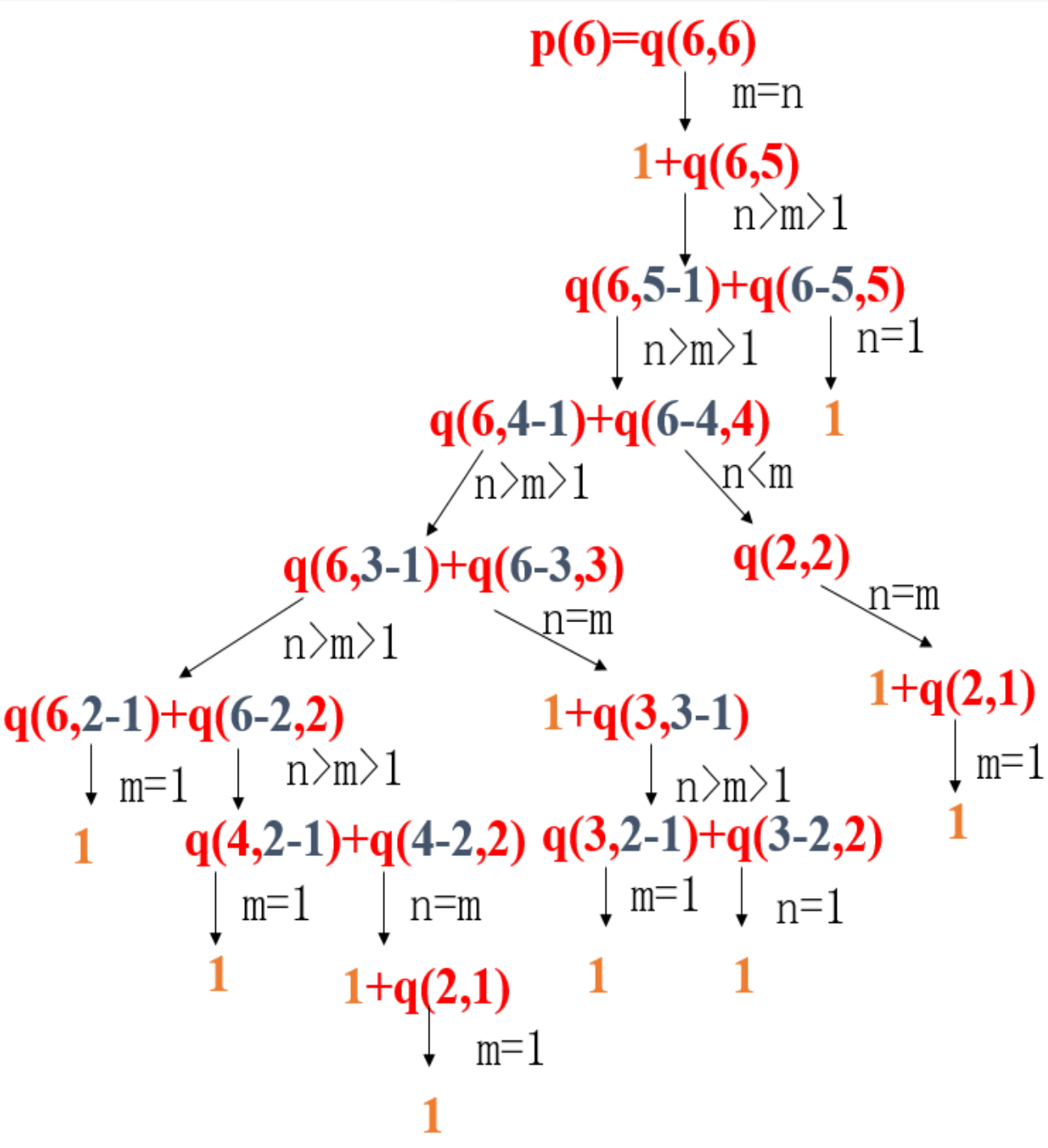
算法设计策略——分治策略

以上关系给出了计算q(n,m)的递归式如下

$$q(n,m)=\begin{cases}1 & n=1 \text{ 或 } m\\ q(n,n) & n<m\\ 1+q(n,n-1) & n=m\\ q(n,m-1)+q(n-m,m) & n>m>\end{cases}$$

- 正整数n的划分数p(n)=q(n,n)。

请自己计算一下p(6)=11



算法设计策略——分治

例6 Hanoi塔问题： 问题

算法思想:

用如下算法移动盘子(记为FourPegsHanoi):

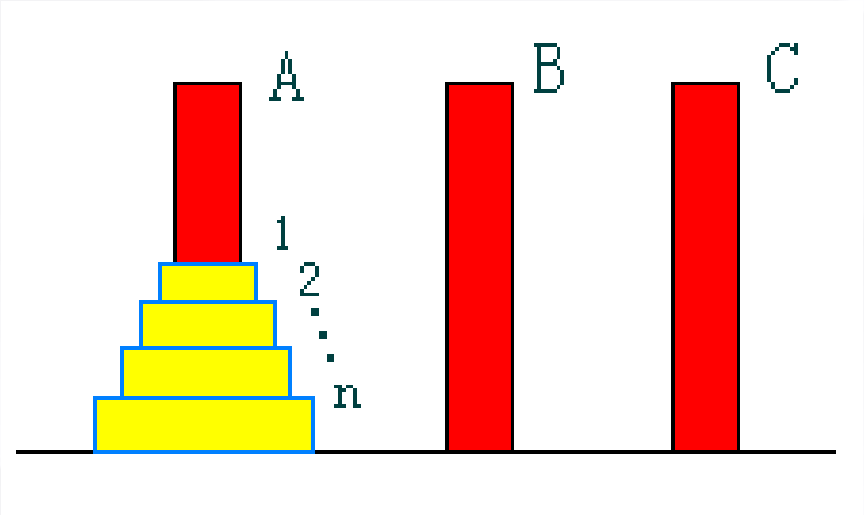
1)、将A柱上n个盘子划分为上下两部分，下方部分共有k(1≤k≤n)个盘子，上方部分共有n - k个盘子。

2)、将A柱上面部分n–k个盘子使用FourPegsHanoi算法经过C、D柱移至B柱。

3)、将A柱剩余的k个盘子使用ThreePegsHanoi算法经过C柱移至D柱。

4)、将B柱上的n–k个盘子使用FourPegsHanoi算法经过A、C柱移至D柱。

```
public static void hanoi(int n, int a, int b, int c)
  {
     if (n =1)  move(a,b)
     else
     {
        hanoi(n-1, a, c, b);
        move(a,b);
        hanoi(n-1, c, b, a);
     }
  }
```



思考题：如果塔的个数变为a,b,c,d四个，现要将n个圆盘从a全部移动到d，移动规则不变，求移动步数最小的方案。

二、递归

递归小结：**优点：结构清晰，可读性强，而且容易用数学归纳法来证明算法的正确性，因此它为设计算法、调试程序带来很大方便。**

缺点：递归算法的运行效率较低，无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。

解决方法：在递归算法中消除递归调用，使其转化为非递归算法。

1.采用一个用户定义的栈来模拟系统的递归调用工作栈。该方法通用性强，但本质上还是递归，只不过人工做了本来由编译器做的事情，优化效果不明显。

2.用递推来实现递归函数。

3.通过Cooper变换、反演变换能将一些递归转化为伪递归，从而迭代求出结果。

后两种方法在时空复杂度上均有较大改善，但其适用范围有限。

算法设计策略——分治策略

分治法所能解决的问题一般具有以下几个特征：

- 1. 该问题的规模缩小到一定的程度就可以容易地解决；
- 2. 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质。
- 3. 利用该问题分解出的子问题的解可以合并为该问题的解；
- 4. 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

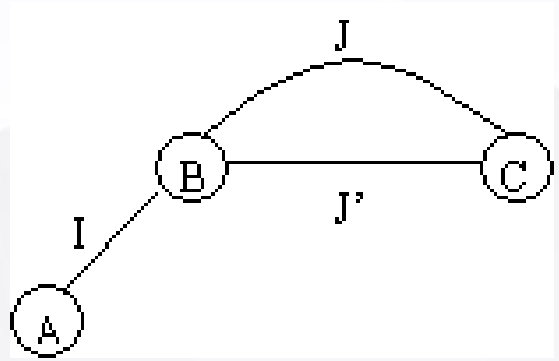
算法设计策略——分治策略

最优化原理（最优子结构性质）：

最优化原理可这样阐述：一个最优化策略具有这样的性质，不论过去状态和决策如何，对前面的决策所形成的状态而言，余下的诸决策必须构成最优策略。简而言之，一个最优化策略的子策略总是最优的。

一个问题满足最优化原理又称其具有*最优子结构性质*。

例如：左图中，若路线I和J是A到C的最优路径，则根据最优化原理，路线J必是从B到C的最优路线。这可用反证法证明：假设有另一路径J’是B到C的最优路径，则A到C的路线取I和J’比I和J更优，矛盾。从而证明J必是B到C的最优路径。



四、分治法的复杂性分析

从分治法的一般设计模式可以看出，用它设计出的算法一般是一个递归过程。因此，分治法的计算效率通常可以用递归方程来进行分析。

为方便起见，设分解阈值$n_0=1$，且算法ADHOC解规模为1的问题耗费1个单位时间。又设分治法将规模为n的问题分成k个规模为n/m的子问题去解，而且，将原问题分解为k个子问题以及用算法MERGE将k个子问题的解合并为原问题的解需用f(n)个单位时间。如果用T(n)表示该分治法Divide-and-Conquer(P)解规模为|P|=n的问题P所需的计算时间，则有:

$$T(n)=\begin{cases} O(1) & n=1 \\ kT(n/m)+f(n) & n>1 \end{cases}$$

当具体参数确定后，我们就可以灵活运用递归方程的求解方法，求出其解，给出渐近表示。

例如，当$n=m^k$时，我们运用迭代法求解上面的方程，有解：$T(n)=n^{\log_m k}+\sum_{j=0}^{\log_m n-1} k^j f(n/m^j)$

四、分治法的复杂性分析

1、adhoc、merge、子问题个数与时间复杂性的关系

对于分治算法来说，其运行时间与adhoc子算法、组合步merge子算法的运行时间以及子问题的个数k有关。

- adhoc子算法的运行时间确定了递归方程的初始值；
- merge子算法的运行时间确定了递归方程的非齐次项；
- 子问题的个数确定了递归方程低阶项的系数。

2、子问题规模与时间复杂性的关系

在分治算法中，规模为n的问题，有可能被分解为k个规模各不相同的子问题，这时，所列出的递归方程用归纳法求解存在一定程度的困难。但是在很多情况下，给定的递归方程类似于某个已知的递归方程，而后者的解是预先知道的。这时，我们就可以猜出所列出的递归方程的解，与已知递归方程的解存在一个常数因子c的关系，然后证明并推导出c的大小，从而证明所列出的递归方程解的上界或者下界。

算法设计策略——分治策略

例2：大整数乘法。问题描述为：有两个n位整数X、Y，计算它们的乘积。基数为b。

方法1：（一般的设计技术，小学就学会的方法）

Y的每一位与X的每一位相乘，再相加。

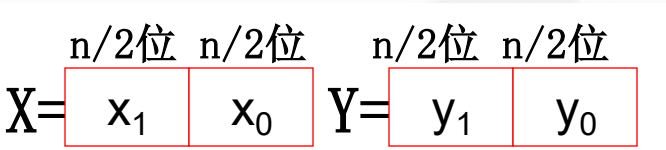
效率：n^2次一位数的乘法，n次移位加法。因此有$O(n^2)$

方法2：（简单的分治设计技术） 设$n=2^k$

分解：X、Y分别分解为两个n/2位的数，即

$X=x_0+x_1b^{n/2}$

$Y=y_0+y_1b^{n/2}$



求解：两个n/2位数的乘法问题；

组合解：$XY=(x_0+x_1b^{n/2})(y_0+y_1b^{n/2})=x_0y_0+(x_0y_1+x_1y_0)\,b^{n/2}+x_1y_1b^n$

算法设计策略——分治策略

如： X=4513 ， Y=2522 ， 基数b=10, n=4=2^2 k=2

方法1：4513×2522=

$$
\begin{array}{r}
9026 \\
9026 \\
22565 \\
+\quad 9026 \\
\hline
\mathbf{11381786}
\end{array}
$$

方法2（采用简单分治策略）：

分解：X、Y分别分解为两个2位的数，即

$X=13+45\times10^2$

$Y=22+25\times10^2$

	n/2位	n/2位		n/2位	n/2位
X=	45	13	Y=	25	22

求解：两个2位数的乘法问题；

组合解：$XY=(13+45\times10^2)(22+25\times10^2)$

$=13\times22+(13\times25+45\times22)\times10^2+(45\times25)\times10^4$

$=286+131500+11250000$

$=11381786$

算法设计策略——分治策略

效率分析：

由于乘法用的时间比加法和移位所用的时间多，只估计乘法的次数：按照我们的策略，计算XY的问题划分为四个子问题，即要作4次两个n/2位数的乘积。

设M_k表示两个2^k位数相乘所需作的一位数乘法的数量：

所以：$M_k=4M_{k-1}$　$M_0=1$

$M_k=4M_{k-1}=4(4M_{k-2})=4(4(4M_{k-3}))=\ldots\ldots=4^kM_0=4^{2k}$

由于 $n=2^k$　　则$k=\log_2 n$

所以 $M(n)=(2^k)^2=(2^{\log 2n})^2=n^2$

即：$\begin{cases} T(1)=1 & n=1 \\ T(n)=4T(n/2)+O(n) & n>1 \end{cases}$

那么，
有没有乘法次数更少
的算法呢?

用master方法解此递归方程：$T(n)=O(n^{\log_2 4})=O(n^2)$

因此，与我们期望的不同，简单的分治法没有达到我们提高性能的目的，即从乘法次数上来看，该算法并没有减少！！

算法设计策略——分治策略

方法3（分治设计技术）： 设$n=2^k$

为了降低时间复杂度，必须减少乘法的次数。为此，改变一下计算 x_0y_0、$x_0y_1+x_1y_0$、x_1y_1 的方式：

$x_0y_0=x_0y_0$

$x_1y_1=x_1y_1$

$x_0y_1+x_1y_0=(x_0-x_1)(y_1-y_0)+x_0y_0+x_1y_1$

效率分析：计算XY要作3次两个n/2位数的乘积和若干次加法和移位；

注： $x_0y_1+x_1y_0$也可以用下面的公式计算： $(x_0+x_1)(y_0+y_1)-x_0y_0-x_1y_1$

算法设计策略——分治策略

设T_k表示两个2^k位数相乘所需作的一位数乘法的数量：所以：$T_k=3M_{k-1}$　$T_0=1$

$T_k=3T_{k-1}=3(3T_{k-2})=3(3(3T_{k-3}))=.....3^kT_0=3^k$

由于　$n=2^k$　$k=\log_2 n$

所以　$T(n)=3^k=3^{\log n}=n^{\log 3}=n^{1.59}$

注：利用对数的性质　$a^{\log_b c}=c^{\log_b a}$

即：$\begin{cases} T(1)=1 \\ T(n)=3T(n/2)+cn \end{cases}$

解此递归方程，有：

$T(n)=O(n^{\log 3})=O(n^{1.59})$

可以看到，这个算法的性能有了比较大的改进！ ☺

有没有更快的方法？

- 如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。
- 最终的，这个思想导致了快速傅利叶变换(Fast Fourier Transform)的产生。该方法也可以看作是一个复杂的分治算法，对于大整数乘法，它能在O(nlogn)时间内解决。
- 是否能找到线性时间的算法？目前为止还没有结果。

算法设计策略——分治策略

例3：矩阵乘法。问题描述为：

已知有两个n阶方阵： $A=(a_{ij})_{n\times n}$ $B=(b_{ij})_{n\times n}$ $i,j=1,2\ldots,n$， 计算它们的乘积$C=AB=(c_{ij})_{n\times n}$

方法1（一般的设计技术-蛮力法）：

$$c_{ij}=\sum_{k=1}^{n} a_{ik}.b_{kj}$$

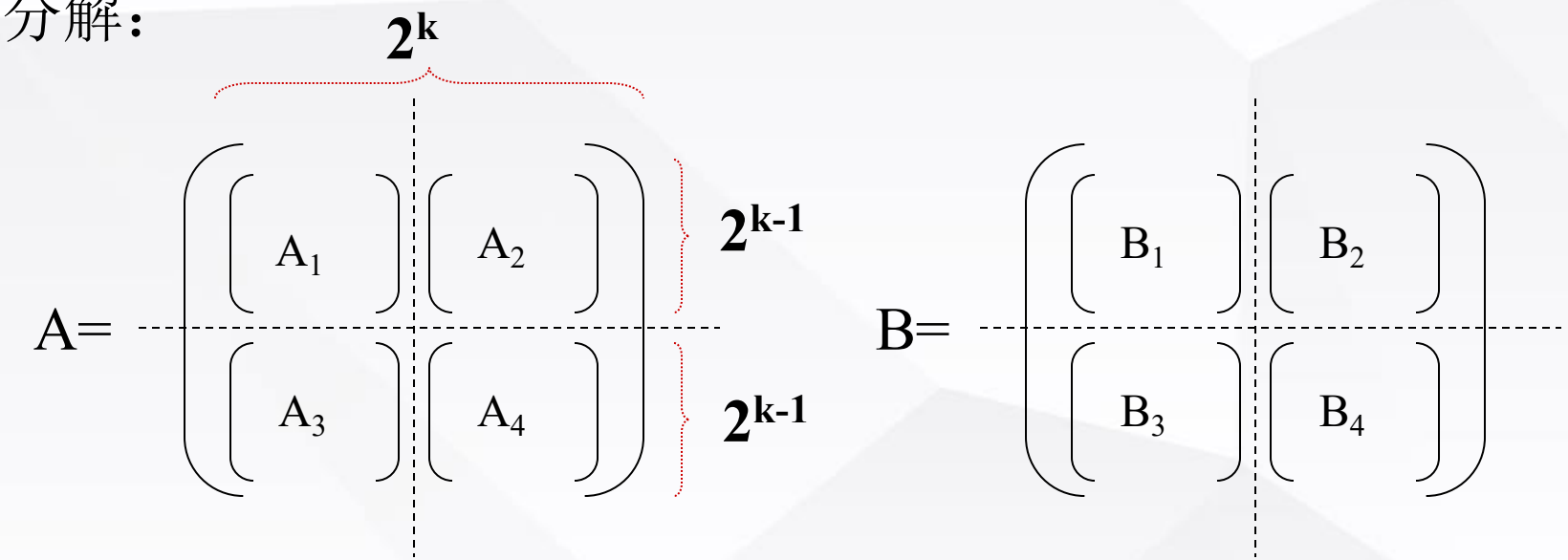
效率： 需做n^3次乘法和若干次$(n^2(n-1))$加法；

有没有少于n^3次乘法的算法呢？？

算法设计策略——分治策略

方法2（分治设计技术——Strassen算法）： 设$n=2^k$

分解：



其中：$A_1,A_2,A_3,A_4,B_1,B_2,B_3,B_4$ 都是2^{k-1}阶的子矩阵。

可以证明，一般的矩阵乘法规则，也适用于分块的矩阵。于是有下面的公式：

算法设计策略——分治策略

若采用下面公式求解：

$$C_1=A_1B_1+A_2B_3$$
$$C_2=A_1B_2+A_2B_4$$
$$C_3=A_3B_1+A_4B_3$$
$$C_4=A_3B_2+A_4B_4$$

$$C=\begin{pmatrix} C_1 & C_2 \\ C_3 & C_4 \end{pmatrix}$$

效率：需做8次$2^{k-1}\times 2^{k-1}$阶矩阵乘法和4次$2^{k-1}\times 2^{k-1}$阶矩阵加法； $O(n^3)$,且由于分割和组合会增加时间！显然，这样分而治之是没有意义的！！

即：
$$\begin{cases} T(2)=b \\ T(n)=8T(n/2)+cn^2 \end{cases}$$

解得：$T(n)=O(n^3)$

Strassen提出了乘法次数少的，7小矩阵分而治之算法！**基本思想是简化矩阵组合步骤。**

算法设计策略——分治策略

Strassen方法：

首先分解并计算出七个小矩阵：

$D=A_1(B_2-B_4)$

$E=A_4(B_3-B_1)$

$F=(A_3+A_4)B_1$

$G=(A_1+A_2)B_4$

$H=(A_1-A_3)(B_1+B_2)$

$I=(A_2-A_4)(B_3+B_4)$

$J=(A_1+A_4)(B_1+B_4)$

组合出结果矩阵C:

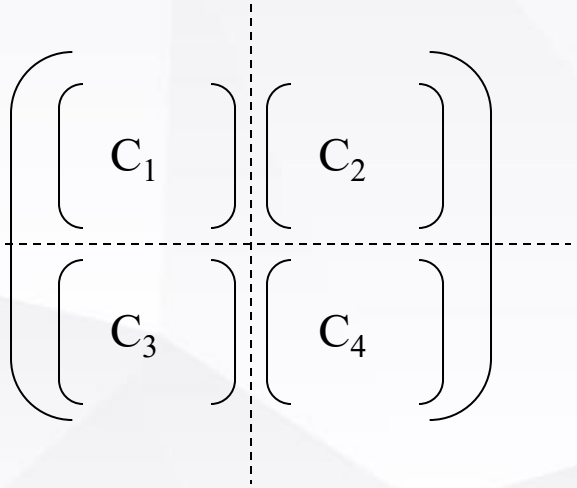
$C_1=E+I+J-G$

$C_2=D+G$

$C_3=E+F$

$C_4=D+H+J-F$

$C=$



算法略！

算法设计策略——分治策略

Strassen方法的效率分析：

计算出七个小矩阵和结果共需要：18次$2^{k-1}\times 2^{k-1}$阶矩阵加（减）法、7次$2^{k-1}\times 2^{k-1}$阶矩阵乘法。 显然，乘法减少了，但加减法增加了。

设P_k表示两个$2^k\times 2^k$阶矩阵相乘所需作的乘法的数量：

所以：$P_k=7P_{k-1}$　$P_1=7$

$P_k=7P_{k-1}=7(7P_{k-2})=7(7(7P_{k-3}))=\cdots\cdots 7^{k-1}P_1=7^k$

由于 $n=2^k$　$k=\log n$

所以 $T(n)=7^{\log n}=n^{\log 7}=n^{2.81}$

即：$\begin{cases} T(2)=b \\ T(n)=7T(n/2)+an^2 \end{cases}$

$$T(n)=O(n^{\log 7})=O(n^{2.81})$$

另外，从空间上看：传统的矩阵乘法，需要存储A，B，C三个矩阵，空间为$3n^2$而Strassen算法的空间需求较多，它除了存储A，B外，还需存储7个小矩阵。设S_k表示两个$2^k\times 2^k$矩阵乘的空间，于是有：$S_k=7S_{k-1}+2(2^k)^2$

解递归方程得到：　$S=O(n^{2.81})$

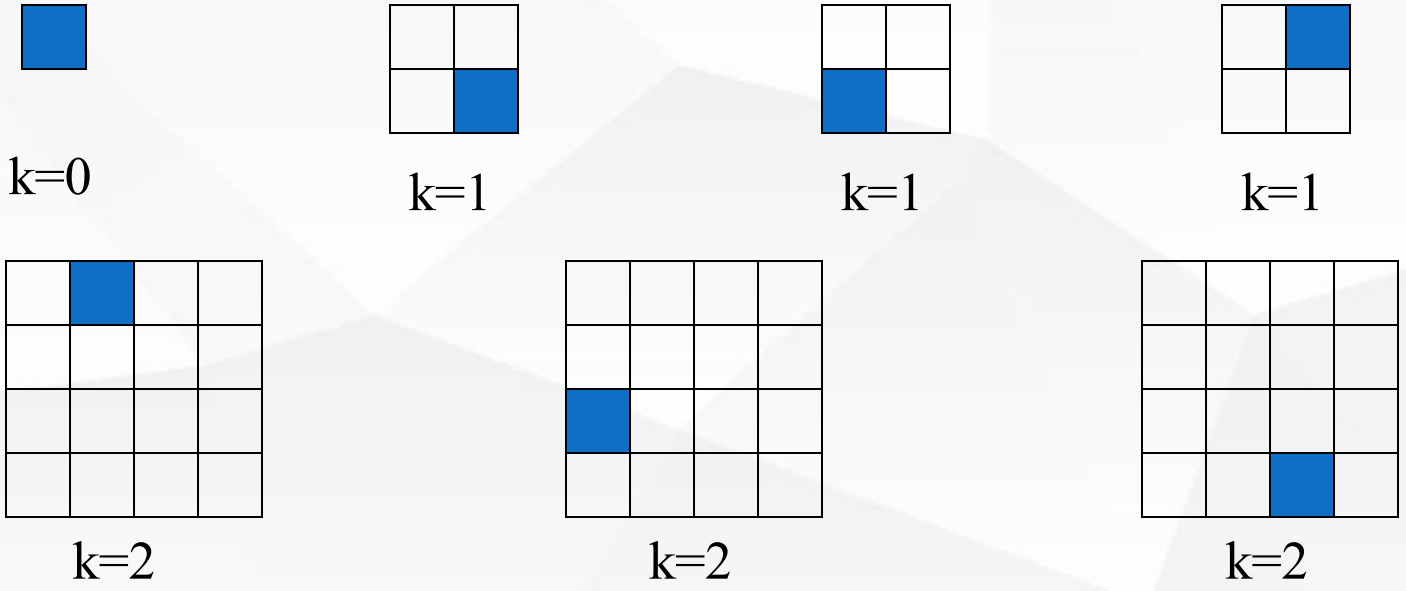
算法设计策略——分治策略

- Hopcroft和Kerr已经证明(1971年)，计算2个 2 × 2 矩阵的乘积，7次乘法是必要的。因此，要想进一步改进矩阵乘法的时间复杂性，就不能再基于计算2×2矩阵的7次乘法这样的方法了。或许应当研究3×3或5×5矩阵的更好算法。
- 在Strassen之后又有许多算法改进了矩阵乘法的计算时间复杂性，分别改为$O(n^{2.795}), O(n^{2.78}), O(n^{2.548}), O(n^{2.494})$等。
- 目前最好的计算时间上界是 $O(n^{2.376})$，最好下界是$\Omega(n^2)$是否能找到$O(n^2)$的算法？目前为止还没有结果。
- 虽然steassen算法把时间代价从$O(n^3)$降至$O(n^{2.81})$，但是进一步得分析指出，后者的系数要比1大很多。当n比较小（例如n<45）且矩阵中非零元素较少时，不宜采用此种方法。

算法设计策略——分治策略

例4：残缺棋盘问题。问题描述为：

残缺棋盘是一个有$2^k \times 2^k$个方格的棋盘，其中恰有一个方格残缺。现在要求用三格板覆盖残缺棋盘，覆盖过程中两个三格板不能重叠，不能覆盖残缺方格，但必须覆盖其它的所有方格。 问如何去覆盖该残缺棋盘（给出覆盖方案）？



算法设计策略——分治策略

不同方向的三角板：

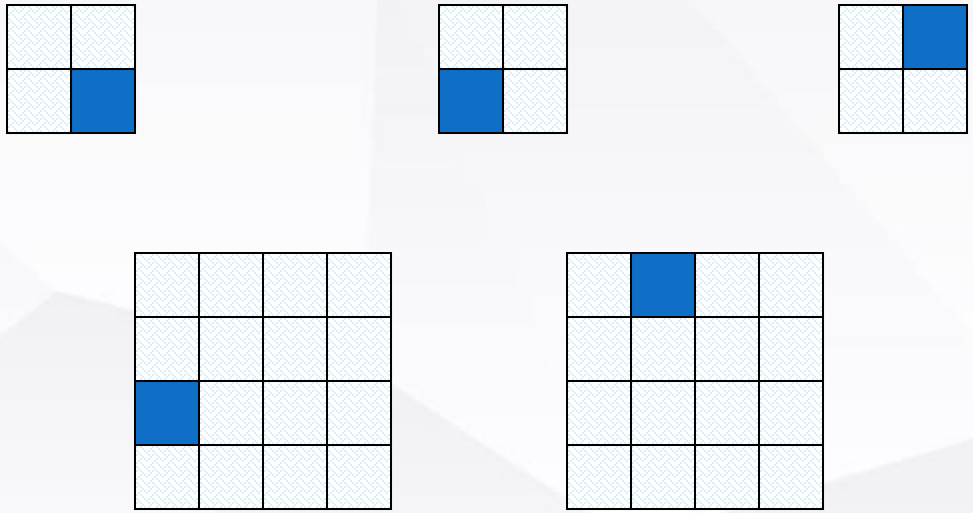


显然：

k=0,不需要三格板；

k=1,需要1块三格板；

k=2,需要5块三格板；



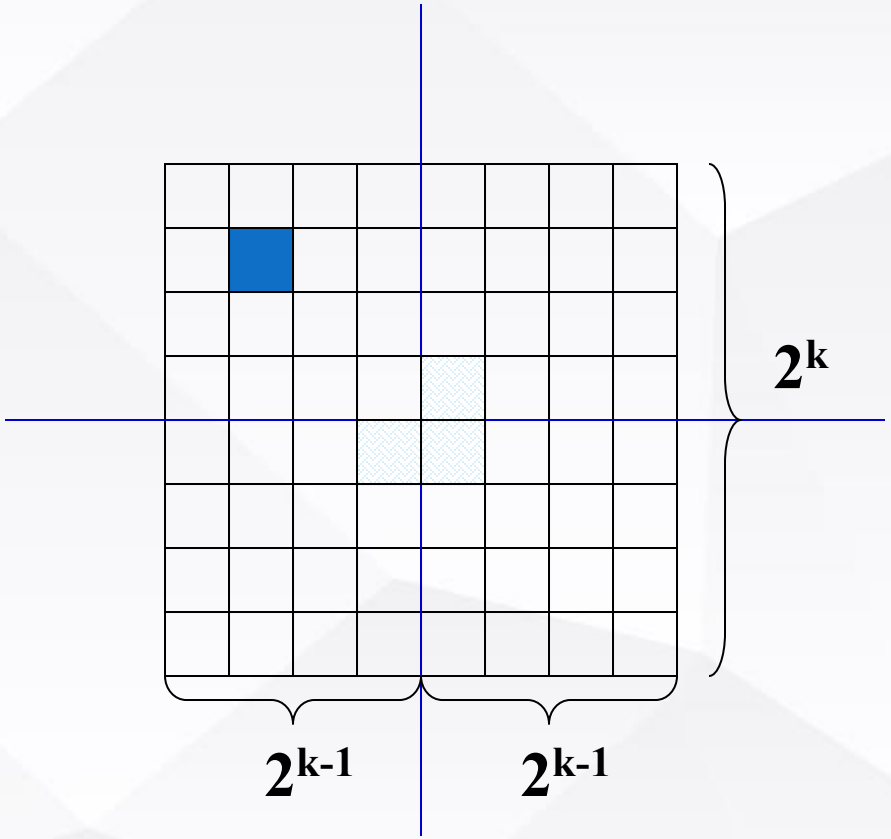
一般结论：

（1）对于任意k,恰好存在2^{2k}种不同的残缺棋盘；

（2）对于任意k，按上面覆盖要求需要（2^{2k}-1）/3块三格板；

算法设计策略——分治策略

分而治之解决方法：



对于k>1：

（1）划分为4个$2^{k-1}\times 2^{k-1}$的残缺棋盘（只有一个包含残块）；

（2）解决3个不包含残块的棋盘覆盖问题；

（3）解决一个包含残块的棋盘覆盖问题；

算法设计策略——分治策略

算法略！

方法的效率：

$$\begin{cases} \text{即：}\ T(k)=d & k=0 \\ \quad\ T(k)=4T(k-1)+c & k>0 \end{cases}$$

解递归方程：$T(n)=O(4^k)=O(n^2)$

课外实习之一：残缺棋盘覆盖仿真，功能包括

（1）自定义棋盘大小

（2）随机产生残缺块位置

（3）用4种不同颜色标识不同的三角板

（4）自动给出覆盖过程（速度可调）

（5）对各种三角板进行自动计数

例5.　分类（排序）问题

分类问题是数据处理中经常使用的一种重要的运算，人们已经设计出了许多很巧妙的分类算法。其中有些算法的分类算法就是根据分治策略设计出来的。比较典型的有归并排序和快速排序。分治法求解排序问题的思想很简单，只需按某种方式将序列分成两个或者多个子序列，分别进行排序，再将已排序的子序列合并成一个有序序列即可。

算法设计策略——分治策略

1、归并分类

归并分类的基本操作是把两个或多个有序的序列合并成一个有序序列。

分治法两路归并排序的思想可以描述为：

（1）如果待排序的序列长度不超过1时，序列就有序了

（2）否则，将待排序的序列一分为二，得到两个长度基本相等的子序列。（分解）

（3）对两个子序列采用同样的方式分别排序。（治之）

（4）将已经排好序的子序列归并成一个有序序列，将子问题的解合并为原问题的解（归并）。（合并）

算法设计策略——分治策略

递归算法描述：

```
void mergeSort(a[], int left, int right)
{
    if (left<right)   //长度超过1则划分成两个子序列
     {  i=(left+right)/2;      //将待排序序列一分为二
        mergeSort(a, left, i);    //对左子序列排序
        mergeSort(a, i+1, right);    //对右子序列排序
        merge(a, b, left, i, right);    //归并到数组b
        copy(a, b, left, right);      //复制回数组a
     }
}
```

主调用：mergeSort(a,1,n)

算法设计策略——分治策略

合并解的子算法(归并算法):merge:

```
void merge(a[], b[],int left,int mid,int right)
{
        i=left;  j=mid+1;
        k=0;
        while((i<=mid)&&(j<=right))
          if(a[i]<=a[j]) b[k++]=a[i++];
          else b[k++]=a[j++];
        while(i<=mid)
          b[k++]=a[i++];
        while(j<=mid)
          b[k++]=a[j++];
}
```

时间复杂性：O(n)

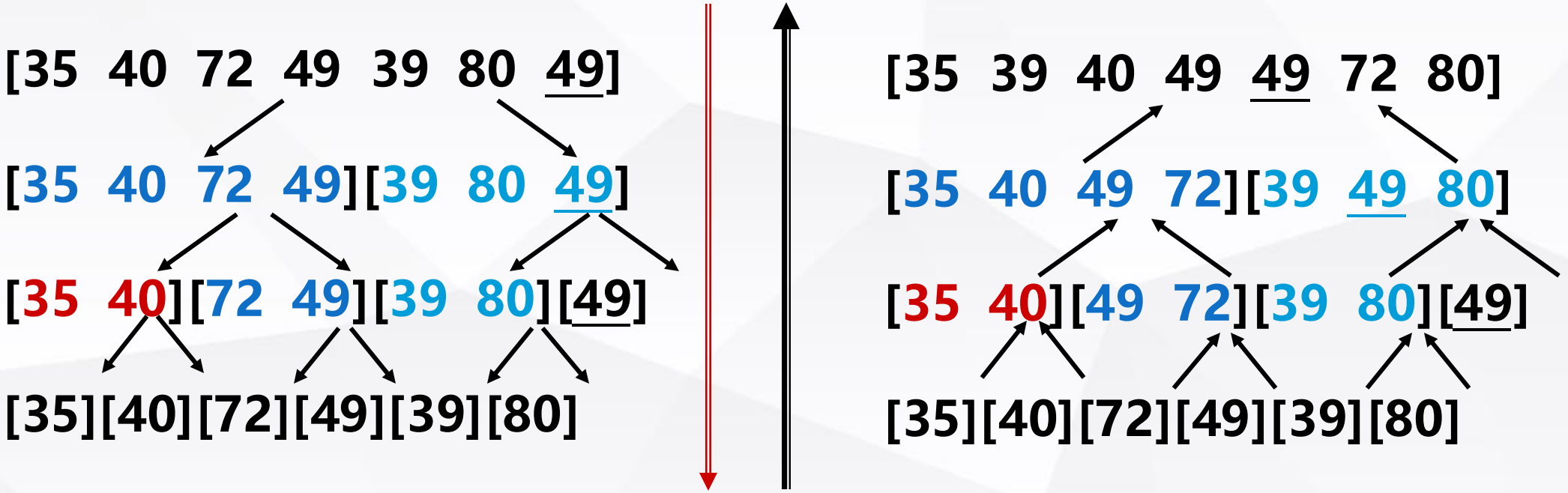
算法设计策略——分治策略

时间复杂性：

$$T(n)=\begin{cases} O(1) & n\le 1 \\ 2T(n/2)+O(n) & n>1 \end{cases}$$

时间复杂性：O(nlogn)　　**空间复杂性O(n)**

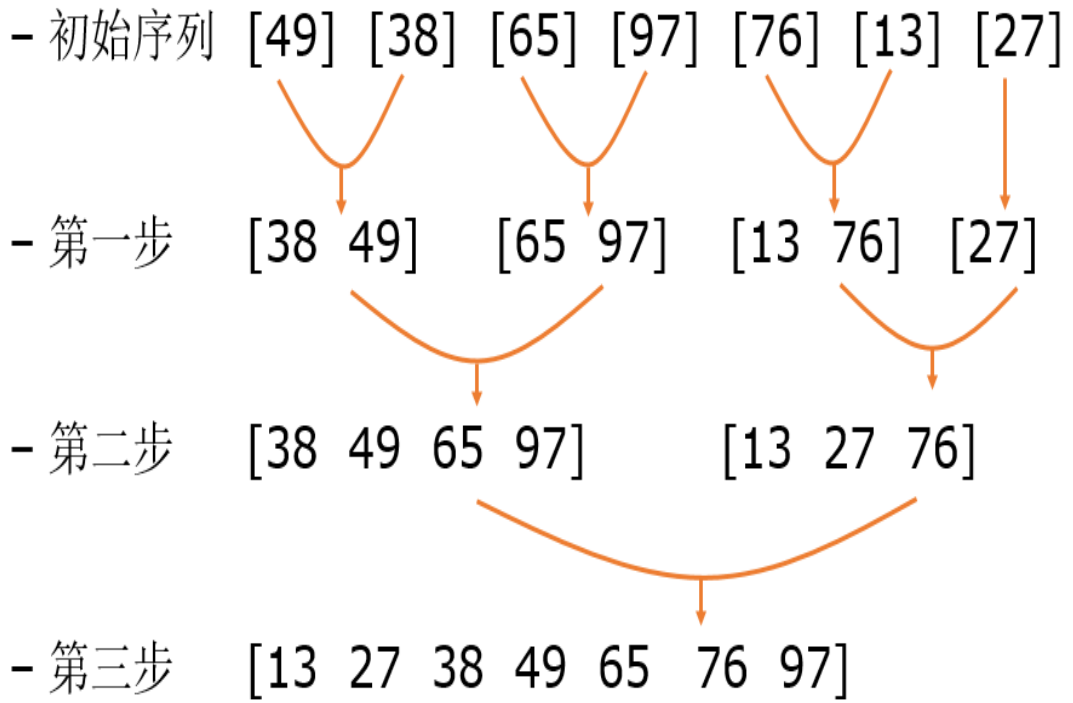
例：



算法设计策略——分治策略

递归的mergeSort可以消除递归：（数据结构中的算法）

```
void mergeSort(a[],int n)
{  t=1;
   while(t<n)
   {  s=t;  t=2*s;  i=0;
      while(i+t<n)       //两两归并
      { merge(a,i,i+s-1,i+t-1,t);
        i=i+t;     }
       if(i+s<n)       //剩下一个不足时
         merge(a,i,i+s-1,n-1,n-i);
    }
}
```



merge算法也要进行简单的修改！

算法设计策略——分治策略

2、快速分类

快速排序是基于分治策略的另一个排序算法。归并算法中使用了一个大小为n的辅助空间，而快速排序不需要额外的空间。其基本思想是：把一个序列就地划分为两个子序列，使第一个序列的所有元素都小于第二个序列的所有元素。不断地这样划分，最后构成n个序列，每个序列只有一个元素，这是整个序列就排好序了。

定义1：给定序列$a_1, a_2, \cdots, a_n$，如果存在元素a_k，使得对所有的i，$1 \le i \le k$，都有$a_i \le a_k$，对所有的j，$1 < j \le k$，都有$a_j \ge a_k$，就称a_k是这个序列的主轴元素，或者叫枢点。

理论上，任何一个元素都可以作为主轴元素。

定义2：指定一个（主轴）元素，重新排列序列，使指定元素成为主轴元素的过程称为一次划分（Partition）。

算法设计策略——分治策略

由分治策略设计出的快速分类算法如下：

（1）分解：确定一个元素，利用一次“划分” 操作使之成为主轴元素，即主轴元素将序列划分为三段：小于主轴元素的前段、主轴元素、大于主轴元素的后段。主轴元素的位置由划分过程确定。从而序列被分解。

（2）递归求解：通过递归调用去求解规模变小的相似的子问题。

（3）合并：两个子序列都已排好序后，不需要执行任何操作则就知道原序列就已排好序。

算法框架如下：

```
void QuickSort (Type a[], int p, int r)
{ if (p<r)
   {  q=Partition(a,p,r);
      QuickSort (a,p,q-1);   //对左半段排序
      QuickSort (a,q+1,r);  //对右半段排序
    }
   }
```

算法设计策略——分治策略

分解/划分算法描述：

```
int Partition (Type a[], int left, int right)
{   int i = left,  j = right;
    Type x = a[left];  //a[left]是要安置的第1个元素
    while (i < j)
    { // 从右向左找小于x的元素
      while(i < j && a[j] >= x)  j--;
      if(i < j)  { a[i] = a[j];          //将小元素放前面
                   i++; }
       // 从左向右找大于或等于x的元素
      while(i < j && a[i] < x)  i++;
      if(i < j) { a[j] = a[i];          //将大元素放后面
                  j--; }
     }
    a[i] = x;
    return i;
}
```

时间复杂性：O(n)

算法设计策略——分治策略

快速排序性能分析：

- 从算法可以看出，如果每次划分后，左右两个子序列的长度基本相同，则快速排序的效率最高，其最好情况时间复杂性为O(nlogn)。反之，如果每次划分后所产生的两个子序列，其中之一为空序列，则快速排序效率最低，其最坏时间复杂性为$O(n^2)$。
- **最坏：** 如果总是选择序列的最左边的第一个元素为主轴元素，则在原始序列正向有序时或者反向有序时达到最坏情况。比较次数=(n-1)+(n-2)+...+2+1+0=n(n-1)/2=$O(n^2)$
- **最好：** 如果在每次划分中，主轴元素恰好是序列的中值，即每次划分得到的左右两个长度几乎相等的子序列，于是有：

$$T(1)=1 \qquad n=1$$
$$T(n)=2T(n/2)+O(n) \quad n>1$$

解得：T(n)=O(nlogn)　　这也是该算法的平均性能！

算法设计策略——分治策略

改善快速排序性能的方法：

（1）改进主轴元素的选择方法。快速排序算法的性能取决于划分的对称性。我们知道，如果以序列的第一个元素作为主轴元素，当序列以递增或递减次序排列时，会出现最坏情况。为了避免此类情况的发生，可以采用以下三种取主轴元素的方式：

- ✓ 取序列的中间元素，即$a_{(left+right)/2}$；
- ✓ 取a_j，j是left～right间的随机整数；
- ✓ 取a_{left}、$a_{(left+right)/2}$、a_{right}三者的中间值；

例如：通过修改算法partition，可以设计出采用随机选择策略的快速排序算法。在快速排序算法的每一步中，当数组还没有被划分时，可以在a[p:r]中随机选出一个元素作为划分基准，这样可以使划分基准的选择是随机的，从而可以期望划分是较对称的。

```
int RandomizedPartition (a[], int p, int r)
{
        i = Random(p,r);
        Swap(a[i], a[p]);
        return Partition (a, p, r);
}
```

算法设计策略——分治策略

改善快速排序性能的方法：

（2）在快速排序中，子序列会变的越来越小，当子序列的长度小到一定程度时，快速排序的速度反而不如一些简单的排序算法（如直接插入法）。因此，当子序列长度小于某个值时可以认为序列足够小，对足够小的序列可采用直接插入排序。

（3）递归算法的效率常常不如相应的非递归算法，为了提高快速排序速度，可以设计出非递归的快速排序算法。基本做法就是使用栈。

算法设计策略——分治策略

归并排序和快速排序在分治策略上的不同：

归并排序和快速排序虽然都运用了分治策略，但是两者的角度不同，因此得到了不同的排序算法：

分解：归并排序的分解过程十分简单，只需将序列一分为二即可；而快速排序的分解方法相对就比较困难（调用“划分” 过程）。

合并：从子问题解得到原问题解的过程看，快速排序非常简单，几乎无须额外的工作，但归并排序则需要复杂一些（调用“归并” 过程）。

特别说明：

（1）分治策略用于算法设计往往需要一些技巧。

（2）表面上看分治就是“分”、“合”，但是效果很明显，原因就在于“分”和“合”的技巧。因此，研究“分”“合”也是分治策略的重点。

一种算法设计策略提供了一种设计算法的启示，对于同一问题，基于同一算法设计策略，算法设计者可以根据各自对问题的理解和分析，提出不同的具体解决方案，设计出不同的算法。

算法设计策略——分治策略

例6. 多项式乘法问题

两个n阶多项式p(x)、q(x)：$p(x)=a_0+a_1x+\ldots+a_nx^n$　　$q(x)=b_0+b_1x+\ldots+b_nx^n$

进行加法或者减法运算，需要n+1次加法或者减法操作，结果还是n阶多项式；运算时间随规模增大而线性增加。

进行乘积运算，将得到一个具有2n+1项系数的2n阶多项式。如果采用一般方法（蛮力）计算，需要$(n+1)^2$次乘法和$n(n+1)$次加法运算。即两个多项式乘法的运算时间则随着输入规模的增大而非线性增加。

分治的思想告诉我们：如果先降低多项式的阶再进行运算，就有可能降低它的运算时间。

算法设计策略——分治策略

（1）分治策略的算法

为了减少两个多项式乘积中的乘法运算次数，采用分治思想，把一个多项式降阶划分成两个多项式：

$p(x)=p_0(x)+p_1(x)x^{n/2}$，$q(x)=q_0(x)+q_1(x)x^{n/2}$

于是有：$p(x)q(x)=p_0(x)q_0(x)+(p_0(x)q_1(x)+p_1(x)q_0(x))x^{n/2}+p_1(x)q_1(x)x^n$

例如：$p(x)=1+x-x^2+2x^3$，$q(x)=1-x+2x^2-3x^3$

$$
\begin{array}{r}
1+x-x^2+2x^3 \\
\times\quad 1-x+2x^2-3x^3 \\
\hline
-3x^3-3x^4+3x^5-6x^6 \\
2x^2+2x^3-2x^4+4x^5 \\
-\ x-\ x^2+\ x^3-2x^4 \\
1+\ x-\ x^2+2x^3
\end{array}
$$

n=4, n/2=2

$p(x)=(1+x)+(-1+2x)x^2$ ：$p0(x)=1+x$　$p1(x)=-1+2x$

$q(x)=(1-x)+(2-3x)x^2$ ：$q0(x)=1-x$　$q1(x)=2-3x$

$p0(x)*q0(x)=(1+x)*(1-x)=1-x^2$

$p0(x)*q1(x)=(1+x)*(2-3x)=2-x-3x^2$

$p1(x)*q0(x)=(-1+2x)*(1-x)=-1+3x-2x^2$

$p1(x)*q1(x)=(-1+2x)*(2-3x)=-2+7x-6x^2$

$p0(x)*q1(x)+p1(x)*q0(x)=1+2x-5x^2$

$p(x)*q(x)=1-x^2+(1+2x-5x^2)x^2+(-2+7x-6x^2)x^4$

$=1+2x^3-7x^4+7x^5-6x^6$

显然，这是一种简单的“分”、“合”！计算的乘法次数并没有减少，还是$O(n^2)$。

算法设计策略——分治策略

（1）分治策略的算法

$$T(2) = O(1) \quad n=2$$
$$T(n) = 4T(n/2)+O(n) \quad n>2$$

求解可得：$T(n) = O(n^{\log 4}) = O(n^2)$

那么，有没有乘法次数更少的方法呢?

$$p(x)q(x) = p_0(x)q_0(x) + (p_0(x)q_1(x) + p_1(x)q_0(x))x^{n/2} + p_1(x)q_1(x)x^n$$

因为：$(p_0(x)+p_1(x))(q_0(x)+q_1(x))$

$= p_0(x)q_0(x)+p_1(x)q_1(x)+p_0(x)q_1(x)+p_1(x)q_0(x)$

所以：$p_0(x)q_1(x)+p_1(x)q_0(x)$

$= (p_0(x)+p_1(x))(q_0(x)+q_1(x))-p_0(x)q_0(x)-p_1(x)q_1(x)$

做下面的变换： 令：$r_0(x)=p_0(x)q_0(x)$

$r_1(x)=p_1(x)q_1(x)$

$r_2(x)=(p_0(x)+p_1(x))(q_0(x)+q_1(x))$

于是有： $p(x)q(x)=r_0(x)+((r_2(x)-r_0(x)-r_1(x))x^{n/2}+r_1(x)x^n$

算法设计策略——分治策略

假设多项式有n项系数，为简单起见，设$n=2^k$(如果n不是2的幂，可以增加0系数项，使其成为2的幂）。这样，我们就可以根据上面的公式把多项式划分成只有n/2项系数的多项式。然后分别计算只有n/2项系数的3个多项式的乘积，再利用上面的公式把3个多项式的乘积合并成一个多项式的乘积。

多项式的划分过程一直进行到只有两个系数为止，这时就直接对只有两个系数的多项式进行计算了。

具体算法略！

（2）复杂性分析

如果多项式有n项系数，假设$n=2^k$,把具有n项系数的多项式划分为只有n/2项系数的两个较小的多项式，原来计算n项系数多项式的乘积，转换为计算3个只有n/2项系数的多项式乘积；另外还有多项式的加减法（多少次自己分析），于是有递归方程：

$$\begin{cases} T(n)=c_1 & n=2 \\ T(n)=3(n/2)+c_2n & n>2 \end{cases}$$

解此递归方程得：

$$T(n)=O(n^{\log 3})=O(n^{1.58})$$

例7. 最接近点对问题

在应用中，常用诸如点、圆等简单的几何对象代表现实世界中的实体。在涉及这些几何对象的问题中，常需要了解其邻域中其他几何对象的信息。例如，在空中交通控制问题中，若将飞机作为空间中移动的一个点来看待，则具有最大碰撞危险的2架飞机，就是这个空间中最接近的一对点。这类问题是计算几何学中研究的基本问题之一。

再如：金断属裂问题，即在金属片上钻n个大小相同的洞，如果洞离的很近，就会断裂，通过计算点的最近距离可以估计金属断裂的概率。

我们着重考虑平面上的最接近点对问题。严格地说，最接近点对可能多于1对。为了简单起见，这里只限于找其中的一对。

问题描述为： 给定n个点 (x_i,y_i) 1<=i<=n，找出其中距离最近的两个点。

$$\sqrt{(x_i-x_j)^2-(y_i-y_j)^2}$$

算法设计策略——分治策略

方法1：（一般算法：蛮力）

共有 n(n-1)/2 对顶点，计算每对顶点间的距离，并比较。 效率为 $O(n^2)$

方法2：（高效算法：分治策略）

将平面上的n个点的集合S分成两个子集S_1和S_2，每个子集中约有n/2个点，然后在每个子集中递归求其最接近点。于是，原问题变为：

（1）求S_1中最近距离的两个点；

（2）求S_2中最近距离的两个点；

（3）由（1）、（2）得到S中的最接近点。

因此，求解的关键：如何实现组合解？

（1）如果组成S中的最近距离点都在S_1中，或都在S_2中，则问题很容易解决！

（2）如果组成S中的最近距离点分别在S_1和S_2中，那么，如何组合解？

容易想到的方法是计算出S_1和S_2中点的距离，需要做$n^2/4$次计算和比较（为什么？），效率与穷举法相同！！

算法设计策略——分治策略

为了好理解，我们先考虑一维的情形：

此时，S中的n个点退化为X轴上的n个实数$x_1,x_2,\ldots,x_n$。最接近点的问题即是找这n个实数中相差最小的两个实数。如何实现？

- 容易想到的一种策略是：

（1）对n个实数排序；—— O(*nlogn*)

（2）进行一次扫描，得到差最小的两个实数；—— O(*n*)

因此，该粗略算法的时间复杂性O(*nlogn*)

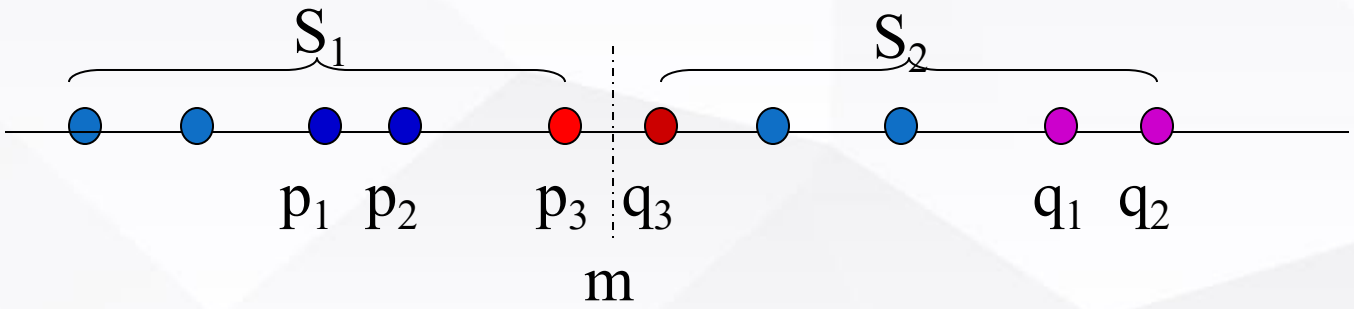
- 用分治技术解决如何解决？

基本的思想就是把实数分为两组，分别求解，再合起来。假设用X轴上的某个点m把S划分为两个集合S_1和S_2，使得$S_1=\{x\in S \mid x\le m\}$ $S_2=\{x \in S \mid x>m\}$，这样，对于所有的$p \in S_1$，$q \in S_2$ 有$p<q$。

算法设计策略——分治策略

我们对S_1和S_2可以分治，得到它们各自的最接近点对$\{p_1, p_2\}$ $\{q_1, q_2\}$，且设$d=min\{|p_1-p_2|,|q_1-q_2|\}$

显然我们知道：S 中的最接近点或者是$\{p_1,p_2\}$，或者是$\{q_1,q_2\}$，或者是某个$\{p_3,q_3\}$,其中$p_3 \in S_1$且$q_3 \in S_2$。

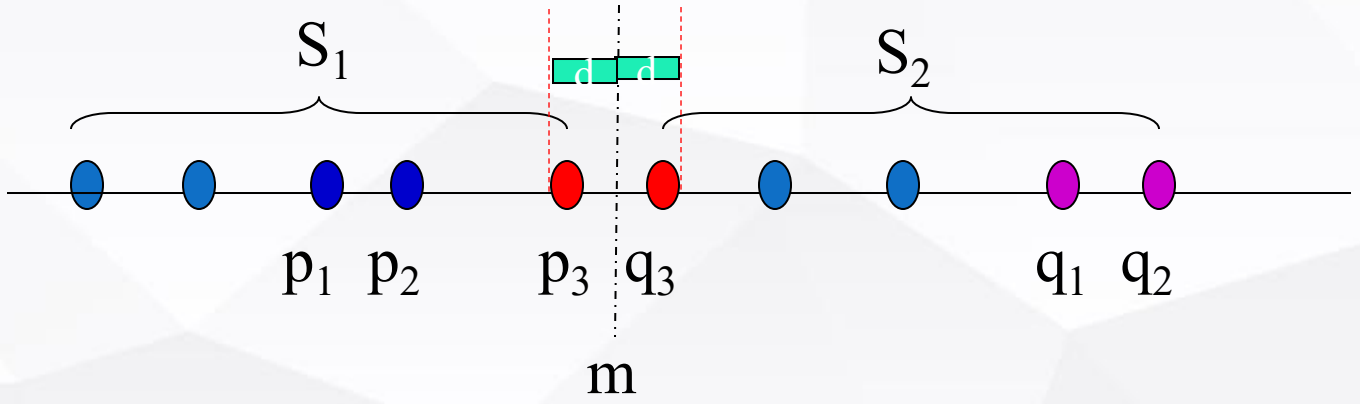


如果S的最接近点对是$\{p_3,q_3\}$,即$|p_3-q_3|<d$，则p_3,q_3两者与m的距离都不超过d，即$|p_3-m|<d$,$|q_3-m|<d$.也就是说$p_3 \in (m-d,m]$,$q_3 \in (m,m+d]$。

算法设计策略——分治策略

于是可以得知：长度为d的半闭区间至多包含S中的一个点，因此，(m-d, m]中至多包含一个S_1中的点。同理， (m, m+d]中也至多包含一个S_2中的点。

进而可知：如果(m-d, m]中有S中的点，则此点就是S_1中的最大点，同理，如果(m, m+d]中有S中的点，则此点就是S_2中最小点。



因此，我们用线性时间就能找到区间(m-d,m]和(m,m+d]中所有点，即p_3和q_3。从而我们用线性时间就可以将S_1的解和S_2的解合并成为S的解。即合并解可以在O(*n*)内完成。

算法设计策略——分治策略

要注意一个问题：即m的取值。因为m的取值会影响到算法的性能。

- 例如，分割点m的选取不当，会造成$|S_i|=1$，$|S_j|=n-1$ $(i+j=1)$的情形，使得$T(n)=T(n-1)+O(n)=O(n^2)$。这种情形可以通过“平衡子问题”方法加以解决：选取各点坐标的中位数作分割点。

例如： m=(max(S)+min(S))/2

算法如下：

```
bool Cpair1(S,d)
 {  n=|S|;
    if(n<2) { d=∞; return false }
    确定m;
    划分构造S1和S2;
     Cpair1(S1,d1);
     Cpair1(S2,d2);
     p=max(S1);    //S1中最大的
     q=min(S2);    //S2中最小的
     d=min(d1,d2,q-p);
     return true;
 }
```

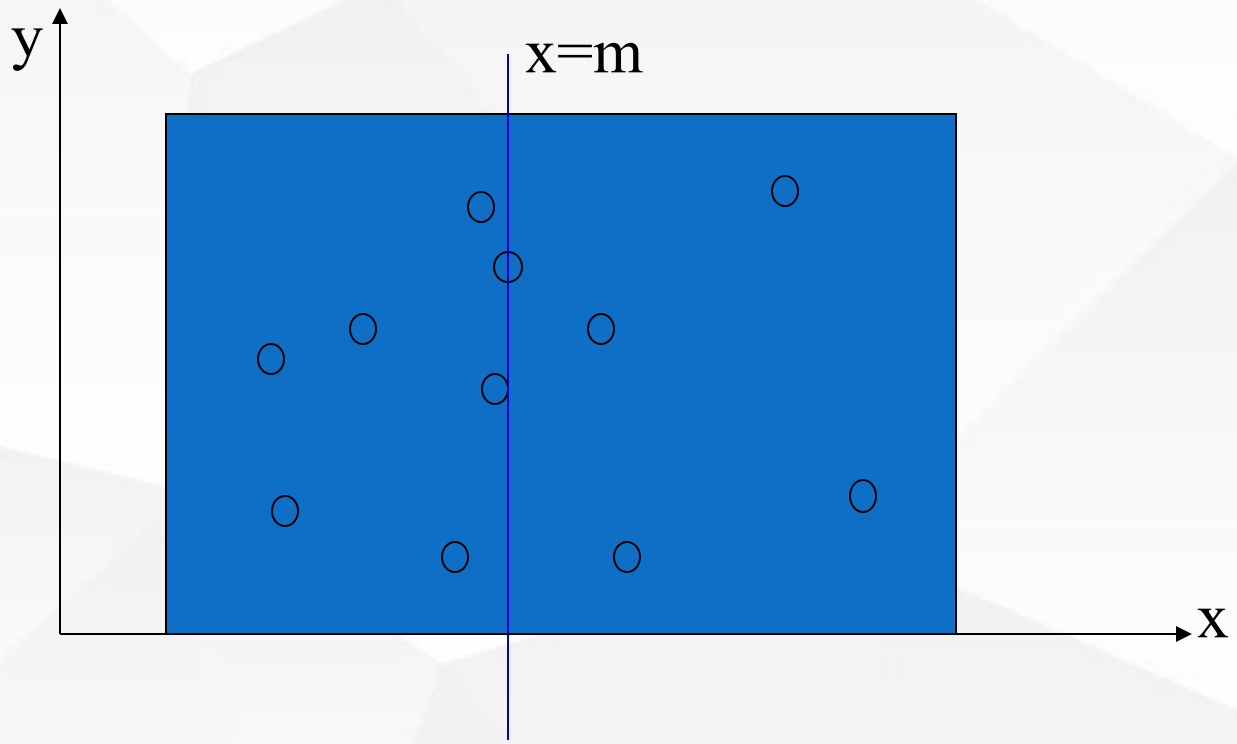
$$T(n)=\begin{cases} O(1) & n<4 \\ 2T(n/2)+O(n) & n\geq 4 \end{cases}$$

T(n)=2T(n/2)+O(n)
=O(nlogn)

算法设计策略——分治策略

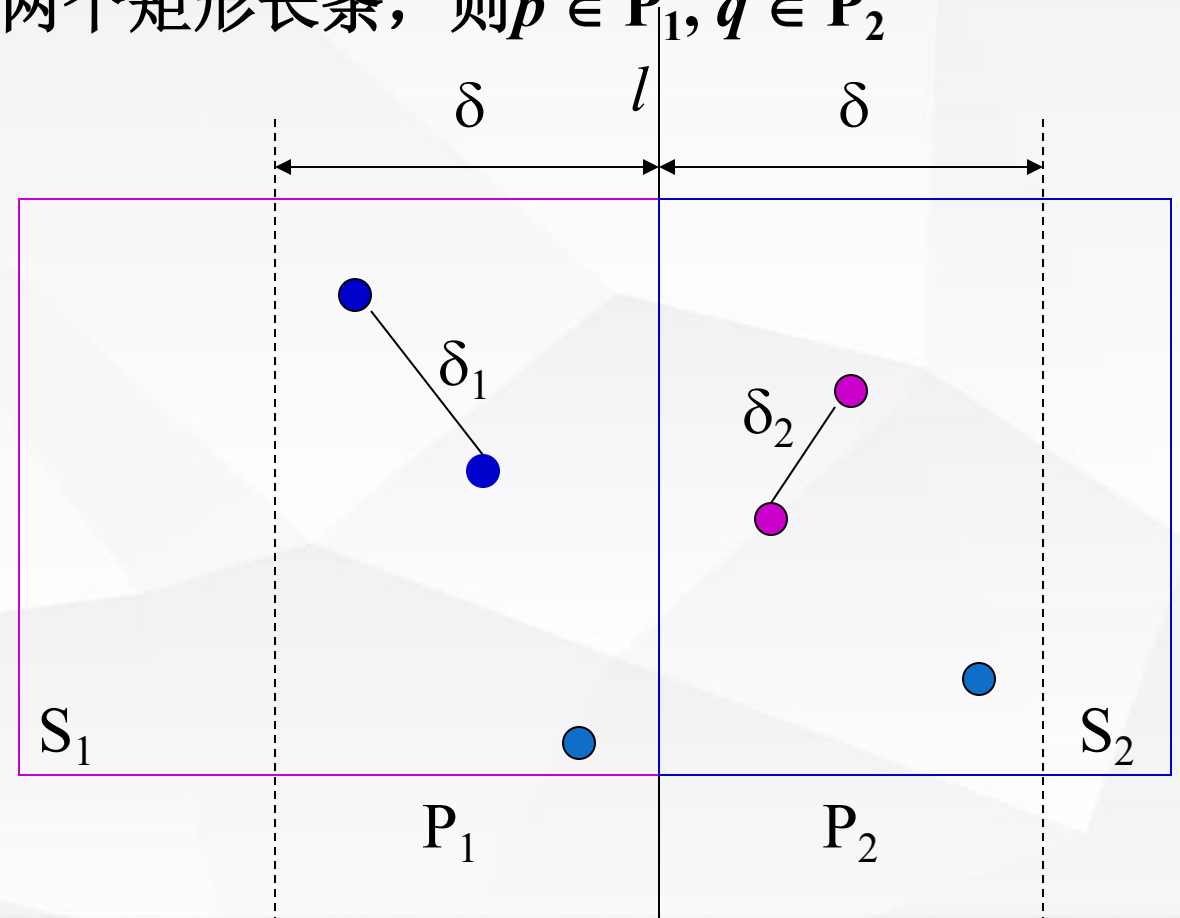
把这个思想推广到二维平面点：

设平面上的n个点为(x_i,y_i)，选取一条直线*l*: x=m作为分割直线，其中m是各点x坐标的中位数。由该直线可以将S划分为两部分：$S_1=\{p\in S|\ x(p)\le p\}$ $S_2=\{p\in S|\ x(p)>p\}$，$S=S_1\cup S_2$。由于m是S中各点x坐标值的中数，S_1和S_2中的点数大致相等。



算法设计策略——分治策略

分别求出S_1和S_2中最近距离点，最近距离分别为δ_1和δ_2，设$\delta=\min(\delta_1, \delta_2)$。于是，S中的最接近点对或者是$\delta$，或者是某个$\{p, q\}$,且$(p, q)$的距离$d(p, q)<\delta$。则，$p$、$q$一定分别属于$S_1$和$S_2$。不妨设$p \in S_1, q \in S_2$，则，$p$、$q$到直线$l$的距离均小于$\delta$。用$P_1$和$P_2$表示直线$l$两侧宽为$\delta$的两个矩形长条，则$p \in P_1, q \in P_2$



算法设计策略——分治策略

显然P_1和P_2中的点为最近点的侯选者，那么，有多少个？我们知道，最多有$n^2/4$个，如果检查这么多个，效率不会提高！如果能否在线性时间内找到p，q，则效率就比较高了。

对于P_1中的任意一点p，它若与P_2中的点q构成侯选最近点，则有$d(p,q)<\delta$，那么P_2中有多少这样的点？

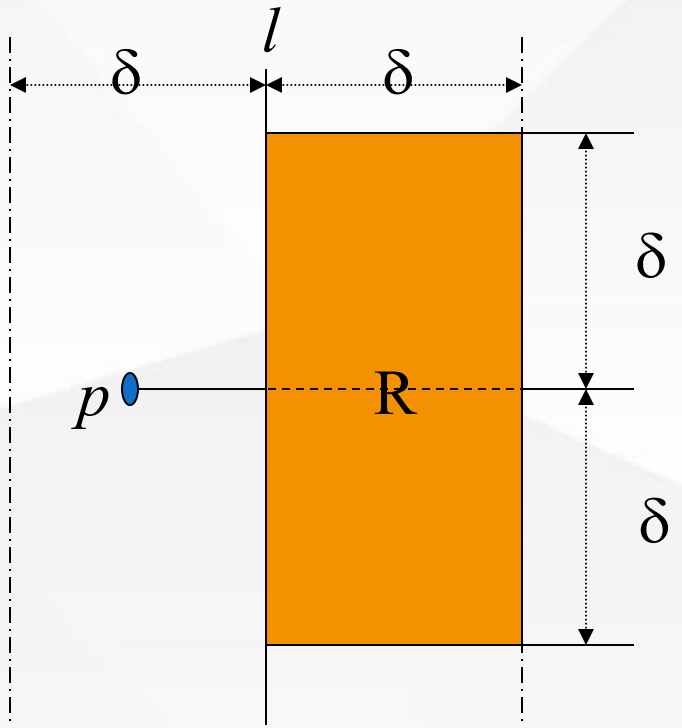
命题：如果存在$p \in P_1, q \in P_2$，使得$d(p,q)<\delta$，那么每个p和q处在距l一个δ的范围内。

证明：假设直线l的方程为$x=x^*$，假设p,q存在，$p=(p_x,p_y)$，$q=(q_x,q_y)$，根据x^*的定义，有：$p_x \le x^* \le q_x$，于是有：

$x^*-p_x \le q_x-p_x \le d(p,q) < \delta$ 和 $q_x-x^* \le q_x-p_x \le d(p,q) < \delta$

因此，每个p,q的x坐标与x^*的差在δ之内，因而与l的距离在δ之内。

同样，可以证明，对y坐标也一样！

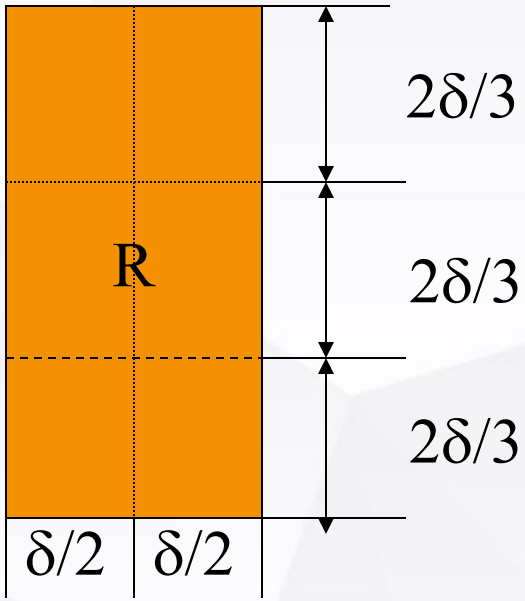


算法设计策略——分治策略

根据这个命题，由δ的含义可以知道，P_2中的任何2个点的距离都不会小于δ，所以可以推出矩形R中最多只有6个S中的点，即可比较的点只有6个！

为什么？

将矩形R的长为2δ的边三等分，将它的长为δ的边二等分，由此导出6个(δ/2)×(2δ/3)的矩形，如下图所示。



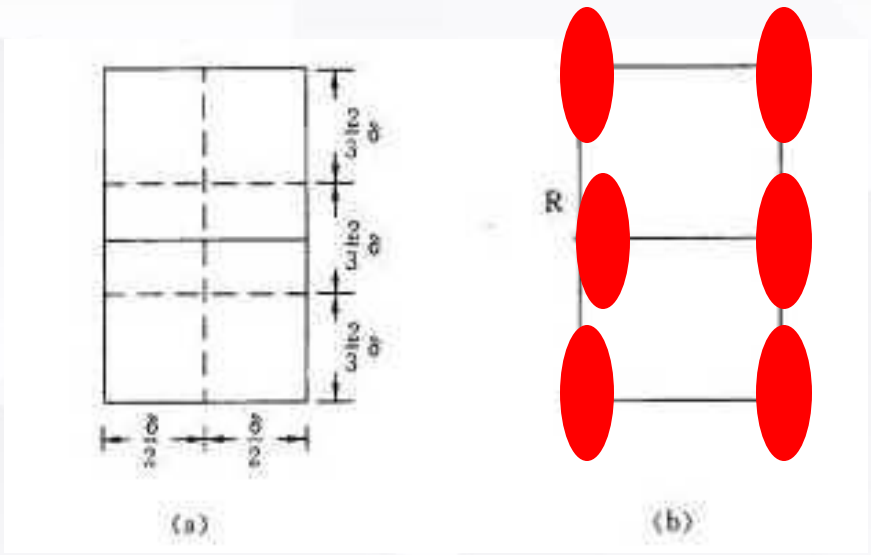
算法设计策略——分治策略

证明：反证法

若矩形R中有多于6个S中的点，则由鸽舍原理易知至少有一个 $\delta/2\times 2\ \delta/3$的小矩形中有2个以上S中的点。设***u,v***是这样2个点，它们位于同一小矩形中，则：

$$(u_x - v_x)^2 + (u_y - v_y)^2 \le (\delta/2)^2 + (2\delta/3)^2 = \frac{25}{36}\delta^2$$

于是***d(u,v)***$\leqslant 5\delta/6 < \delta$ 。这与 δ 的意义相矛盾。也就是说矩形R中最多只有6个S中的点。右图是矩形R中含有S中的6个点的极端情形。



算法设计策略——分治策略

因此，对于P_1中任一点p，P_2中最多只有6个点与它构成最近点的侯选者，于是组合解只需要检查$6\times(n/2)=3n$对侯选点而不是$n^2/4$个。—— 点的数量级降了！

但是还有一个问题：不知道是哪6个点，如果要找出这6个点还要去逐个判断的话，效率并没有提高！ 所以必须能快速找到这6个点！

方法是：将p和P_2中的所有S_2的点投影到垂直线l上，由于能与p点构成侯选点的点都在矩形R中，所以它们在直线l上的投影点距p在l上的投影点的距离都小于δ。从上面的分析知，这种点只有6个。因此，若将P_1和P_2中的所有点按其y坐标排好序，则对P_1中的所有点，对排好序的所有点作一次扫描，就可以找出所有最接近点对的侯选者，对P_1中的每一点最多只要检查P_2中排好序的相继的6个点。

算法设计策略——分治策略

算法如下：

算法的效率：

$$T(n)=\begin{cases} O(1) & n<4 \\ 2T(n/2)+O(n) & n\geq 4 \end{cases}$$

- T(n)=O(nlogn)

```
public static double CPair2(S)
{  n=|S|;
   if(n==1) return(∝);
   if(n==2) return(S中这两点的距离);
   m=S中各点x间坐标的中位数;
   S1={p∈S|x(p)<=m }; //构造S1和S2;
   S2={p∈S|x(p)>m };
   d1=cpair2(S1);
   d2=cpair2(S2);
   dm=min(d1,d2);
   设P1是S1中距分割线l的距离在dm之内的所有点组成的集合;
      P2是S2中距分割线l的距离在dm之内所有点组成的集合;
   将P1和P2中点依其 y 坐标值排序;
 并设X和Y是相应的已排好序的点列;
   通过扫描X以及对于X中每个点检查Y中与其距离在dm之内的
所有点(最多6个)可以完成合并;
   当X中的扫描指针逐次向上移动时，Y中的扫描指针可在宽为
2dm的区间内移动;
   设dl是按这种扫描方式找到的点对间的最小距离;
   d=min(dm,dl);
   return d;
}
```

例8. 选择问题(select problem)

选择问题是指在*n*个元素的集合中，选出某个元素值大小在集合中处于第 *k* 位的元素，即所谓的求第 *k* 小元素问题（*kth_smallest*）

这类问题与排序问题、比赛问题可以归属同一类，只是具体要求有区别。它们都是以*n*个某全序集上的元素作为输入，排序问题是求*n*个元素的全顺序；而选择问题是求第k个元素，对前*k-1*元素的顺序不关心，对后*n-k*个元素的顺序也不关心。当*k=1*时，就是最小元素问题；当 *k=n*是就是求最大元素问题；当*k=n/2*时就是中值问题（在实际应用中经常用到，例如中值滤波）

算法设计策略——分治策略

（1）简单算法

最容易想到的是：直接使用排序算法，排序后取第*k*个即可。最好的排序算法的时间复杂度为0(*nlogn*)。

另一个容易想到的算法就是快速排序中的划分方法，即求划分位于位置 *k* 的划分。

```
RandomizedSelect(Type a[],int p,int r,int k)
{
    if (p==r) return a[p];
    i=RandomizedPartition(a,p,r),
    j=i-p+1;   //位置是相对于p的
    if(j==k) return a[i]     //划分结束
    else if (k<j) return RandomizedSelect(a,p,i-1,k);
    else return RandomizedSelect(a,i+1,r,k-j);
}
```

算法设计策略——分治策略

该算法与前面快速排序的划分方法类似，其区别是在划分后，不是对划分元左、右两侧的部分同时递归，而是只选其中k所在的一侧递归。

算法复杂性：可以证明，算法RandomizedSelect可以在$O(n)$平均时间内找出n个输入元素中的第k小元素。但在最坏情况下，算法randomizedSelect仍需要$O(n^2)$计算时间。

那么有没有在最坏情况下时间复杂度仍然是$O(n)$的算法呢？我们知道，之所以会有最坏情况，是因为划分无法避免划分后主轴元素位于两端的极端情况。因此，我们就从这里入手，精心挑选主轴元素，以保证每次划分所得的两个子序列的大小相对接近，即即使不能保证每次恰在中间，也要保证较小一半占有“一定的”比例。

（2）线性时间的选择算法

➢基本思路：如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的2个子序列的长度都至少为原序列长度的ε倍（$0<\varepsilon<1$是某个正常数），那么就可以在最坏情况下用$O(n)$时间完成选择任务。

例如，若 $\varepsilon=9/10$，算法递归调用所产生的子序列的长度至少缩短1/10。所以，在最坏情况下，算法所需的计算时间$T(n)$满足递归式 $T(n)\leq T(9n/10)+O(n)$ 。由此可得$T(n)=O(n)$。

算法设计策略——分治策略

➢一种划分方案:(二次取中法)

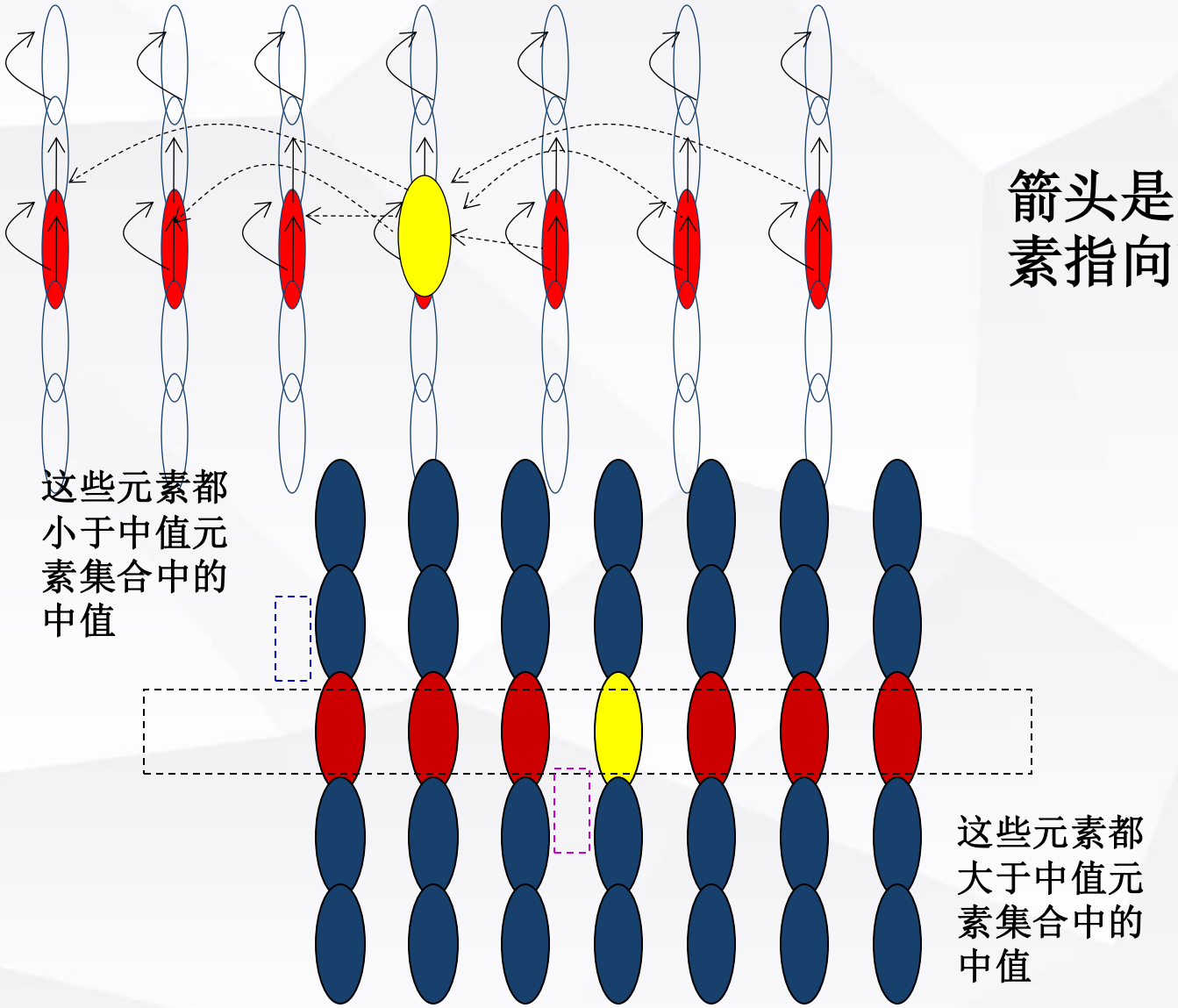
（1）把n个元素划分为$\lceil n/5 \rceil$组，每组5个元素，最后一组可能不足5个元素（用最大值补足,也可以忽略）。

（2）求出每组的中值。用任意一种排序算法，将每组中的元素排好序，得到每组的中值，于是得到具有$\lceil n/5 \rceil$个中值元素组成的集合M。

（3）求中值的中值。可递归求出M中的中值m^*，如果$\lceil n/5 \rceil$是偶数，就找它的两个中位数中较大的一个。

以这个元素作为划分基准——主轴元素。

算法设计策略——分治策略



箭头是由较大元素指向较小元素

算法设计策略——分治策略

➢分治：

以m^*为主轴元素，把元素序列划分为两个子序列，左边的子序列的元素不大于m^*，右边的子序列的元素不小于m^*。然后根据 k值的大小决定如何递归。

算法见参考书，略！

▪说明：

为了分析简化，设所有元素互不相同，轴元素为x。

（1）因为在每一组中有2个元素小于本组的中位数；而$\lfloor n/5 \rfloor$个中位数中又有$\lfloor (n-5)/10 \rfloor$个小于轴元素x，于是，至少有$3\lfloor (n-5)/10 \rfloor$个元素比轴元素$x$要小。

（2）同样道理，至少有$3\lfloor (n-5)/10 \rfloor$个元素比轴元素大。

（3）当$n \geq 75$时，$3\lfloor (n-5)/10 \rfloor \geq n/4$，所以按此轴元素划分所得的2个子序列的长度都至少缩短$1/4$。

算法设计策略——分治策略

效率分析：

我们在这里只给出结论，**T(n)**由三部分组成： **T(n) ≤T(n/5)+T(3n/4)+cn**

其中，**T(n/5)**是在**n/5**个中间值中求二次中间值的时间，**T(3n/4)**是在其中一个子序列中继续求解的时间，**cn**包括对**n/5**组求每组的中间值及一趟划分操作的时间。递归方程如下：

$$T(n)\le\begin{cases}C_1 & n<75\\ C_2n+T(n/5)+T(3n/4) & n\ge 75\end{cases}$$

C_1 为直接简单排序时间
C_2n 为执行循环的时间
解递归方程得： **T(n)=O(n)**

算法设计策略——分治策略

给我们的启示：

- 线性选择算法的设计过程体现了分治策略用来优化算法设计的威力；
- 改进算法的一种策略是对原有的算法进行分析，在其次要部分付出一定的代价，使其在主要部分获得收益；
- 选择问题的线性算法的研究吸引了一些学者专家的注意，取得了一些研究成果。主要是减小了线性函数的常数。“五元组”算法的具体结论是$T(n) \leq 16n$；V.Praat等设计的算法是$T(n) \leq 5.43n$；目前最好成果是$T(n) \leq 2.95n$。
- 在实际应用中，算法比较复杂，而且n比较大才有效。
- 中值问题和选择问题的关系：选择问题在$k=n/2$时就是中值问题。中值问题是选择问题的一个特例，解决中值问题的算法的复杂度应该更低。
- 对中值问题，目前最好的结果是：对于较大的n，至少需要比较$2n$次。这个值与最好的选择算法的时间复杂度$T(n) \leq 2.95n$已经很接近了。
- 中值问题作为选择问题的特例，在实际应用中很常用。例如图形图象处理中的中值滤波就是一个非常重要的、非常有效的滤波算法。

算法设计策略——分治策略

分而治之技术总结：

1、思想很简单；

2、同样是分之，但分解的方法可能不同；

3、解决小问题是容易的；

4、组合解有时很容易，有时很难；

5、简单的分解和组合，算法的效率与传统的效率是一样的；

6、经验告诉我们：研究组合解的方法可以探索好的算法！

很多著名的算法都是采用分而治之技术设计的：

二分查找；

归并分类；

快速分类；

算法设计策略——分治策略

递归是典型的分治技术，能够运用好递归有时会有柳暗花明的感觉。

设计递归算法的关键：

分析问题，得到问题的递归性描述！

改进递归算法效率的方法关键是化为非递归后的“栈优化”

算法设计策略——分治策略

- 自己理解和尝试用分治策略解决下面问题：

1、任给整数序列：a1..an，设计算法找出其中总和最大的区段（如果有多个，长度短的优先），并分析算法的时间复杂性。

例如：A[1..19]={1,-2,7,2,6,-9,5,6,-12,-8,13,0,-3,1,-2,8,0,-5,3}

总和最大是17，有三个区段，最短的两个，长度6

7,2,6,-9,5,6　　13,0,-3,1,-2,8

2、有一实数序列 $a1, a2, a3, \ldots, an$，若$i<j$ 且 $aj>ai$，则(ai,aj)构成了一个逆序对，请设计算法，求序列中的逆序对个数，并分析算法的时间复杂性。

例如：序列 (4,3,2)，逆序对有：(4,3)，(4,2)，(3,2)，共3个。

算法设计策略——分治策略

例：最大连续子序列和

- 给定一个含n（$n \geq 1$）个整数的序列，要求求出其中最大连续子序列的和。
- 序列（-2，11，-4，13，-5，-2）的最大子序列和为20。
- 序列（-6，2，4，-7，5，3，2，-1，6，-9，10，-2）的最大子序列和为16。
- 规定一个序列最大连续子序列和至少是0，如果小于0，其结果为0。

算法设计策略——分治策略

解

a_{low} … a_i … a_{mid} | a_{mid+1} … a_j … a_{high}

maxLeftSum　　maxRightSum

（a）递归求出maxLeftSum和maxRightSum

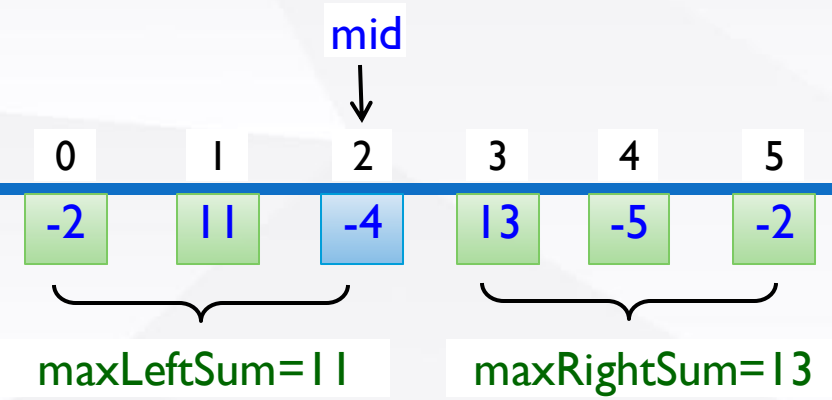
maxLeftBorderSum + maxRightBorderSum

a_{low} … a_{mid} | a_{mid+1} … a_{high}

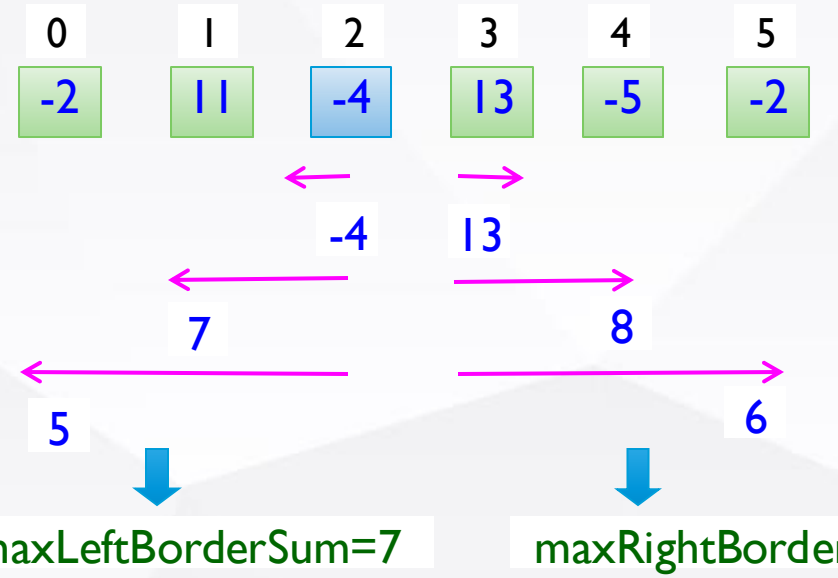
（b）求出maxLeftBorderSum+maxRightBorderSum

max3(maxLeftSum,
maxRightSum,
maxLeftBorderSum+maxRightBorderSum)

（c）求出a序列中最大连续子序列的和



（a）递归求出maxLeftSum和maxRightSum



（b）以-4为中心的最大连续子序列和为20

（c） 结果=max3(11，13，20)=20

算法设计策略——分治策略

```
//求出3个int整数中的最大值
int max3(int a,int b,int c)     {
    return max(max(a,b),c);
}
//递归算法：求a序列中最大连续子序列和
int maxSubSum3(vector<int>&a) {
    int n=a.size();
    return maxSubSum31(a,0,n-1);
}
```

```
//被maxSubSum3调用
int maxSubSum31(vector<int>&a,int low,int high)
{ if (low==high)          //子序列只有一个元素时
    {   if (a[low]>0)    //该元素大于0时返回它
            return a[low];
        else             //该元素小于或等于0时返回0
            return 0;
    }
```

```
int mid=(low+high)/2;                          //求中间位置
int maxLeftSum=maxSubSum31(a,low,mid);   //求左子序列之和
int maxRightSum=maxSubSum31(a,mid+1,high); //求右子序列之和
    int maxLeftBorderSum=0,lowBorderSum=0;
    for (int i=mid;i>=low;i--)                          //求a[i..mid]
    {   lowBorderSum+=a[i];
        if (lowBorderSum>maxLeftBorderSum)
            maxLeftBorderSum=lowBorderSum;
    }
    int maxRightBorderSum=0,highBorderSum=0;
    for (int j=mid+1;j<=high;j++)                       //求a[mid+1..j]
    {   highBorderSum+=a[j];
        if (highBorderSum>maxRightBorderSum)
            maxRightBorderSum=highBorderSum;
    }
    int ans=max3(maxLeftSum,maxRightSum,
                 maxLeftBorderSum+maxRightBorderSum);
    return max(ans,0);}
```

$T(n)=1$ 当$n=1$

$T(n)=2T(n/2)+n$ 当$n>1$

$\Rightarrow T(n)=O(n\log_2 n)$

END